

$\rightarrow$ label 1
`mov eax, ebx`
 $\vdots$

Size of jump: -128, 127 (you cannot jump too many bytes)

3) Pointer arithmetic

$\rightarrow$ which arithmetic operations make sense on pointers?

$\rightarrow$ any op. that makes sense (3 operations) $\leftarrow$

ex: adding a constant: $a[7] = \overset{\text{in C}}{*(a+7)} \checkmark = \overset{\text{in C}}{*(7+a)} = 7[a]$ in assembly

subtracting: $a[-4] ? \checkmark$

• multiplying two pointers: $a * b$ X

• dividing two pointers: X

• adding two pointers: X $[a+b]$

• subtracting two pointers: $\checkmark$

Pointer = offset (address)

So, only 3 operations make sense on pointers:

scalar $\Leftarrow$ 1) Subtracting 2 pointers: $a - b \checkmark \rightarrow$ helps us calculate the length of sth.

pointer $\Leftarrow$ 2) Adding a numerical constant: $a[7] \checkmark$

pointer $\Leftarrow$ 3) Subtracting a numerical constant: address - ct. $(a[-7]) \checkmark$
 $\downarrow$
 $*(a-7)$ in C

~~$p+2 = ??$~~ allowed in NASM, but not correct.

4) Dereferentiating a pointer: [] dereferencing a pointer (asm.)
 $*$ (in C)

$\rightarrow$ constant, the offset starting from the beginning of the data segment.

`r db 17`

`add EDX, [EBX + ECX * 2 - r - 7]` ? X syntax error "invalid effective address".

`add EDX, [EBX + ECX * 2 + r - 7]` ? $\checkmark$ (we can add a constant)

`mov [EBX + ECX * 2 + a + b - 7], BX` ? X

mov [EBX + ECX * 2 + 0 - 6 - 7], BX ? ✓

$$\text{Offset} = [\text{base}] + [\text{index} * \text{scale}] + [\text{ct}]$$

↓ 8 reg. ↓ 7 reg. ↓ 1, 2, 4, 8
(-ESP)

indirect addressing

direct addressing

loaded in memory
⇒ beginning of the seg
⇒ for address
⇒ selector
load time

execution time

registers

mov EAX, [EBX] ✓

mov EAX, EBX ✓

mov EAX, [2 * EBX + 7] ✓

mov EAX, [EAX + 4 * EBX + 17] ✓

mov EAX, 23 X → I need to deref → mov EAX, [23] ✓

mov EDX, [ECX + 2 * ESP + 7] X → can't use ESP.

mov EAX, [EBX * 2] ✓ → only 1, 2, 4, 8 are valid indexes.

? mov EAX, [EBX * 3] X → but asm will do mov EAX, [EBX + EBX * 2]

push dword [EAX * 9 + 12] X → but asm will: push dword [EAX + EAX * 8 + 12]

mov EAX, EBX ⊖ EDX X → "invalid effective address"

↳ I cannot in assembly time

mov EAX, 2-4 ✓

CMP / TEST

5) Assembly language basics:

- 5.1. • machine language → sequences of bytes → set of machine instructions → CPU.

• assembly language → programming language in which the basic instructions set corresponds with the machine operations

↓
Symbolic language

mnemonic + label

(shortcut) ex: instead of addition → "add".

5.2. BASIC ELEMENTS: → the value associated with it is an offset.

→ labels = user-defined NAMES for pointing to DATA

... report the underlying action

- instructions = mnemonics which suggest the what of action
- directives = indications given to the assembler
- location counter = integer, read-only, managed by the assembler
 - ↳ count the nr. of bytes starting from the beginning of the data segment to the current variable location?

labels: a db 2
mov AL, [a]
mov AL, [002]

every segment has its own location counter
'\$'

directive: resb

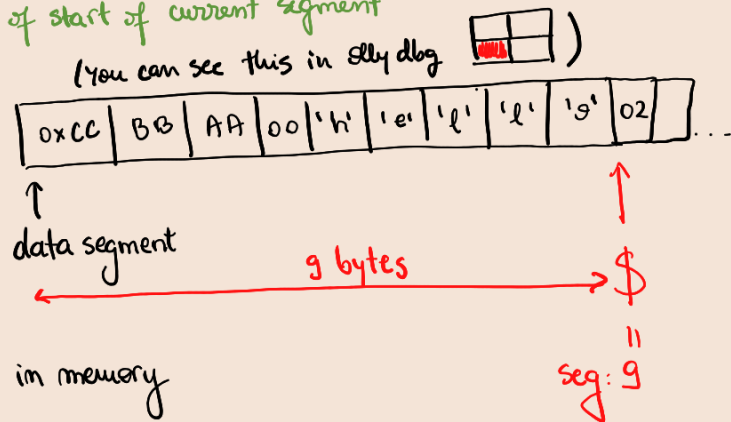
instruction: add

location counter: NASM

- '\$' - "address of here"
- '\$\$' - length of the data segment → POINTER, ADDRESS
- '\$\$' - start of the current segment/section → POINTER, ADDRESS
- "address of start of current segment"

data ...
[a] db 'hello' dd 0xAABBCC
db 'h','e','l','l','o'
data-segment-size equ \$-\$\$
db 2

CONSTANT → not stored in memory



5.3. line format

[label[:]] [prefixes] [mnemonic] [operands] [; comment]

here: jmp here ; label + mnemonic + operand + comment

repz cmpsd ; prefix + mnemonic + comment

start: ; label + comment

a dw 2, 42h ; label + mnemonic + 2 operands + comment

VALID CHARS for LABELS:

[A-Z]

[a-z]

[0-9]

! ,

VALID VARIABLE NAME:

should start with a letter, -, ?

IDENTIFIERS are CASE-SENSITIVE

var ≠ VAR

IMPLICIT NAMES are NOT CASE-SENSITIVE

'.'
'?'
'\$'
'#'
'@'

mov = MOV

eax = EAX

LABELS { code } the value associated with
 { data } it is an offset.

